

Lernskript: DTO-Pattern & Information Hiding

Ein Data Transfer Object (DTO) ist ein einfaches Objekt, das ausschließlich dazu dient, Daten zwischen Schichten oder über eine API zu transportieren. Es enthält keine Geschäftslogik — nur Daten.

Teil 1: Die grundlegende Analogie - Das "Warum"

- **Konzept:** Stell dir vor, du bist Crew Chief auf einer großen Tour. Dein internes Tourplan-Dokument enthält alles: Gagen, Hotelrechnungen, Sicherheitscodes, interne Absprachen. Dieses Dokument gibst du niemals an die Öffentlichkeit weiter.

Was du dem Veranstalter gibst, ist ein **öffentlicher Flyer** — er enthält nur das, was er wissen darf: Showzeit, Setlänge, technische Anforderungen.

Die **Entity** (z.B. `Task.java`) ist dein internes Tourplan-Dokument — vollständig, mit allen internen Feldern. Das **DTO** (z.B. `TaskDto`) ist der öffentliche Flyer — nur das, was nach außen darf.

- **Das Problem ohne DTO:**
 - Die Entity enthält interne Felder wie `createdAt`, die für den API-Aufrufer irrelevant oder sicherheitskritisch sind.
 - Änderungen an der Datenbankstruktur brechen sofort die öffentliche API.
 - Controller und Client kennen direkt die interne Datenbankstruktur — das verletzt das Prinzip der **Separation of Concerns**.
-

Teil 2: Praktische Anwendungsfälle - Das "Wofür"

Use Case 1: Die Entity — das interne Dokument

- **Zweck:** Repräsentiert eine Datenbanktabelle. Enthält alle Felder — auch interne.
- **Code-Beispiel:**

```
@Entity
@Table(name = "tasks")
@Getter @Setter @NoArgsConstructor
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String description;
    private boolean completed = false;
    private LocalDateTime createdAt; // internes Feld – gehört nicht in die API!

    @PrePersist
```

```
protected void onCreate() {  
    this.createdAt = LocalDateTime.now();  
}  
}
```

Use Case 2: Das DTO als Record — der öffentliche Flyer

- **Zweck:** Definiert exakt, was der API-Aufrufer sieht. `createdAt` wird bewusst ausgelassen.
- **Warum ein Record?** Records sind immutable (unveränderlich) — ein DTO wird nie verändert, nur gelesen. Der Compiler erzwingt das automatisch.
- **Code-Beispiel:**

```
/**  
 * Data Transfer Object for the Task entity.  
 * Represents the public API view – decoupled from the database model.  
 * The field createdAt is intentionally excluded (internal database concern).  
 */  
public record TaskDto(Long id, String description, boolean completed) {}
```

Use Case 3: Das Mapping im Service — Entity zu DTO

- **Zweck:** Der Service ist die einzige Schicht, die weiß wie Entity und DTO zusammenhängen. Der Controller kennt nur das DTO. Die Datenbank kennt nur die Entity.
- **Code-Beispiel:**

```
public List<TaskDto> getAllTasks() {  
    return taskRepository.findAll()  
        .stream()  
        .map(task -> new TaskDto(  
            task.getId(),  
            task.getDescription(),  
            task.isCompleted()  
        ))  
        .toList();  
}
```

- **Was passiert hier:**
 1. `findAll()` gibt `List<Task>` zurück — Entities mit allen Feldern.
 2. `.stream().map(...)` wandelt jede Entity in ein TaskDto um.
 3. `createdAt` wird beim Mapping einfach weggelassen — **Information Hiding**.

Teil 3: Vertiefung (JavaMasta's Profi-Tipps)

- **Information Hiding:** Das Weglassen von `createdAt` im DTO ist kein Zufall — es ist eine bewusste Architekturentscheidung. Der Client soll nur das sehen, was er braucht. Dieses Prinzip heißt Information Hiding und ist ein Kernprinzip des Clean Code.

- **SRP im DTO-Pattern:** Die Entity hat eine Aufgabe (Datenbankabbildung), das DTO hat eine andere (API-Kommunikation). Der Service hat die dritte (Mapping und Geschäftslogik). Jede Klasse eine Verantwortung — das ist das **Single Responsibility Principle**.
- **POST gibt die Entity zurück — GET gibt das DTO zurück:** In unserem Taskmaster-Projekt gibt `createTask()` die gespeicherte Entity zurück (damit der Aufrufer die generierte id sieht). `getAllTasks()` gibt DTOs zurück. Das ist eine bewusste, vertretbare Entscheidung — in professionellen Projekten würde man auch für POST ein Response-DTO erstellen.
- **MapStruct:** In größeren Projekten wird das manuelle Mapping durch die Bibliothek **MapStruct** ersetzt. Sie generiert den Mapping-Code automatisch zur Compile-Zeit aus einer simplen Interface-Definition. Das Prinzip bleibt dasselbe.